

Note: The Project title is slightly different from the one proposed. Logic remains the same.

Weather-Induced Price Volatility in a High-Value Floriculture Market

AI/ML and Computer Science Project using Daily Shevanti Prices (2010–2025)

Central Project Question

Do short-term weather conditions leave measurable signatures in daily Shevanti prices, and can AI/ML models use those weather signals to improve price estimation and identify weather-associated price shocks?

This guide tells you what to build, how to organize the work, what analyses to perform, and what you must demonstrate at each review. The immediate goal is a strong, reproducible student project. The results can later be developed into a journal paper.

1. Project Overview

Shevanti is highly perishable. Flower availability can respond quickly to rainfall, temperature, humidity and other short-term weather conditions. When supply changes rapidly while demand remains present, daily market prices may move sharply. Your task is to investigate this relationship computationally using fifteen years of daily price and weather data.

Important boundary

This project focuses on WEATHER and PRICE. Do not make festivals, auspicious days, seasonal decomposition, ARIMA/SARIMA, or deep-learning time-series forecasting the main topic.

2. Project Objectives

1. Build a clean daily dataset that combines Shevanti prices with weather observations for the same dates.
2. Identify which weather variables and short-term weather histories are associated with unusual price movements.
3. Discover weather-related price signatures, such as price behaviour after heavy rain or unusually hot/humid conditions.
4. Build and compare machine-learning models with and without weather information.
5. Determine whether weather information improves prediction especially on high-volatility or weather-stress days.
6. Use explainable AI to show which weather factors influenced individual predictions.
7. Build a small interactive Weather–Price Intelligence application to explore historical relationships and model estimates.

3. What You Are Trying to Find Out

- Does heavy rainfall tend to be followed by higher or lower Shevanti prices?
- Is the effect immediate, or does it appear one, two, three or more days later?
- Do several consecutive rainy days matter more than a single rainy day?
- Do unusually high or low temperatures correspond to abnormal prices?
- Does humidity provide useful information beyond rainfall and temperature?
- Which weather variables are most useful to an ML model?
- Does adding weather information reduce prediction error compared with using recent prices alone?
- On which days does the model fail, and what can those failures teach us?

4. Expected Inputs

Dataset	Minimum fields	Notes
Daily Shevanti prices	Date, Price	2010–2025; same price definition/unit throughout if possible
Daily weather	Date, Rainfall, Temperature,	Use one consistent location/source where

	Humidity	possible
Optional weather	Wind speed, solar radiation or other variables	Include only if reliable and relevant

Every external weather dataset must have its source recorded. Do not mix stations or measurement definitions without documenting the change.

5. Phase 1 — Understand and Validate the Data

Do not begin machine learning immediately. First establish that the price and weather data are trustworthy and aligned.

- Check start and end dates and the number of observations.
- Identify missing dates, duplicate dates, missing values and impossible values.
- Check whether price units or weather units change anywhere in the dataset.
- Plot the complete price series and each major weather variable.
- Flag extreme prices and extreme weather values for investigation; do not automatically delete them.
- Prepare a short Data Quality Report describing all issues and how each was handled.

Do not treat every extreme value as an error

A very high Shevanti price or very heavy rainfall may be exactly the type of event this project needs to study. Verify unusual observations before deciding what to do with them.

6. Phase 2 — Build the Master Weather–Price Dataset

Merge the price and weather records using the date. The result should contain one row per day.

Date	Price	Rainfall	Max Temp	Min Temp	Mean Temp	Humidity
Day 1	...	...	...	...	...	...
Day 2	...	...	...	...	...	...

- Use consistent units for the entire period.
- Keep the original raw files unchanged; create a separate processed dataset.
- Create a data dictionary describing every field, unit, source and any cleaning performed.
- Keep a research log of important decisions.

7. Phase 3 — Create Weather Features

Today's price may respond not only to today's weather but also to weather during the previous few days. Your feature-engineering task is to represent that short-term weather history in a way the ML model can learn from.

Feature group	Examples to create	Purpose
Current-day weather	Today's rainfall, temperature, humidity	Represents current conditions
Lagged weather	Weather 1, 2, 3 and up to 7 days earlier	Tests delayed effects
Accumulated rainfall	Total rainfall over previous 3 and 7 days	Represents sustained wet conditions
Consecutive conditions	Number of consecutive rainy days	Distinguishes one rainy day from prolonged rain
Weather stress flags	Heavy rain / unusually hot / unusually humid	Allows event-style analysis
Recent price history	Recent daily prices and short moving summaries	Provides the price-only baseline

Start with a small, interpretable feature set. Add features only when you can explain what they represent. Avoid creating hundreds of features simply because it is possible.

8. Phase 4 — Define Weather Events

To discover weather-related price signatures, convert important weather conditions into clearly defined events. Examples include heavy-rain days, multi-day rain episodes, unusually hot days or high-humidity periods.

- Define event categories using the distribution of your own weather data and, where possible, domain or meteorological guidance.
- Document exactly how each event is defined.
- Ensure each event category has enough historical examples to analyse.
- Do not choose thresholds merely because they produce attractive results.

Recommended starting point

Begin with rainfall because it is intuitive and likely to produce a clear student workflow. Once the rainfall pipeline works, repeat the same approach for temperature and humidity.

9. Phase 5 — Build a Weather Event Window Engine

Develop a reusable program component that takes a weather event date and extracts prices around that date. A practical starting window is seven days before to seven days after the event.

- For each heavy-rain event, retrieve the surrounding daily prices.
- Normalize the price behaviour relative to a nearby non-event baseline so different years can be compared.
- Combine repeated events to obtain a typical response pattern.
- Record when the price response begins, when it peaks and when it returns toward normal.

Concept: Weather Price Signature

A weather price signature is the typical pattern of price movement around a defined weather event. For example, prices may remain normal on the rain day, rise one or two days later, and return toward baseline after several days.

10. Phase 6 — Discover and Compare Weather Price Signatures

Generate separate signatures for the main weather conditions. Do not assume that every weather variable matters.

Weather condition	Questions to answer
Heavy rainfall	Is there a delayed price response? How long does it last?
Consecutive rainy days	Is prolonged rainfall associated with a stronger response?
High temperature	Are unusual heat conditions associated with abnormal prices?
High humidity	Does humidity add information beyond rainfall?
Combined weather stress	Do combinations produce stronger signatures than individual conditions?

A useful output is a Weather Signature Database containing the event type, number of historical events, typical response timing, peak response and recovery period.

11. Phase 7 — Build the Machine-Learning Experiment

The main AI/ML question is not “Which algorithm wins?” The main question is whether WEATHER INFORMATION adds useful predictive information.

Experiment	Inputs	Purpose
Model A — Simple baseline	Recent price information	Establish a basic reference
Model B — Price-only ML	Recent price-derived features	Measure what ML can learn from prices alone
Model C — Weather-aware ML	Recent prices + weather features	Test the value added by weather

Use a small number of suitable models. A simple regression baseline, Random Forest and XGBoost are sufficient for this project. Do not compare ten or fifteen algorithms.

12. Phase 8 — Evaluate the Models Correctly

- Keep data in chronological order. Do not randomly mix future years into the training data.
- Use earlier years for training, later years for validation and the most recent years as an untouched final test set.
- If possible, add walk-forward evaluation after the basic pipeline is working.
- Report MAE and RMSE, and optionally a suitable percentage-based error metric.
- Evaluate performance on all days AND separately on weather-event/high-volatility days.
- Compare Model B and Model C carefully. This comparison answers whether weather features are genuinely useful.

Critical result

A small improvement on ordinary days but a large improvement on heavy-rain or weather-stress days may be more important than a tiny improvement in overall average error.

13. Phase 9 — High-Volatility / Price-Shock Detection

As an optional second AI task, classify whether a day shows an unusually large price movement. The exact definition of a “price shock” should be chosen after inspecting the historical distribution rather than using an arbitrary number.

- Create a clear normal-day versus price-shock label.
- Compare a price-only classifier with a weather-aware classifier.
- Use evaluation measures suitable for uncommon events, such as precision, recall, F1 and PR-AUC.
- Do not rely on accuracy alone because most days may be normal.

14. Phase 10 — Explainable AI

Use SHAP or another suitable explainability method after the primary model is validated. The goal is to understand why the model made a prediction, not merely to display a feature-importance chart.

- Identify which weather variables are most influential overall.
- Examine whether recent rainfall, accumulated rainfall or temperature is more informative.
- Explain several individual high-price predictions.
- Explain several model failures as well as successes.
- Check whether the model is relying too heavily on recent prices and barely using weather; that is an important finding too.

15. Phase 11 — Similar Historical Weather Situations

Add a simple similarity module. Given a selected weather scenario, retrieve historically similar days or weather episodes and show what happened to Shevanti prices afterward.

- Start with a simple nearest-neighbour approach using normalized features.

- Show three to five comparable historical situations.
- Display their weather conditions and observed price behaviour.
- Use this as an interpretable companion to the ML prediction.

16. Phase 12 — Weather–Price Intelligence Application

Build a small Streamlit application only after the analysis and models are working. Keep the application focused on demonstrating the research.

Page	What it should show
1. Historical Explorer	Select a date; show actual price, weather conditions and surrounding price movement.
2. Weather Event Explorer	Choose heavy rain / heat / humidity; show historical event signatures and examples.
3. Price Scenario Estimator	Enter recent price information and weather conditions; show a model estimate or estimated range.
4. Why This Estimate?	Show the most important factors influencing the selected prediction.
5. Similar Historical Cases	Show comparable historical weather situations and their price outcomes.
6. Model Performance	Compare price-only and weather-aware model performance, including event days.

Do not overclaim the application

The tool is a research prototype. It should present model estimates and historical evidence, not guarantee a future market price. A price range is preferable to false precision when uncertainty can be estimated.

17. Recommended Technology Stack

Task	Recommended tools
Data processing	Python, Pandas, NumPy
Visualization	Matplotlib, Plotly
Machine learning	Scikit-learn, XGBoost
Explainability	SHAP
Application	Streamlit
Version control	Git and GitHub

18. Division of Work

Weather data validation and source documentation	Price feature engineering and ML-ready dataset
Weather feature engineering	Baseline, Random Forest and XGBoost models
Weather-event definitions	Chronological validation and model evaluation
Event-window engine	Weather-feature ablation experiments
Weather price-signature discovery	Price-shock classification extension
Similar historical weather cases	SHAP and local explanations
Weather Event Explorer UI	Price Estimator and Model Performance UI

Both students jointly own data integration, GitHub/reproducibility, experiment design, application integration, interpretation, final presentation and final report.

19. Fourteen-Week Roadmap

Week	Main work	Required output
1	Understand problem, inspect price and weather data	Repository + initial EDA
2	Clean price data and document issues	Price Data Quality Report
3	Clean/validate weather data and sources	Weather Data Quality Report
4	Merge datasets and create data dictionary	Master dataset v1
5	Create lagged/accumulated weather features	Feature dataset v1
6	Define weather events and build event-window engine	Reusable event module
7	Generate rainfall price signatures	Rainfall analysis
8	Extend signatures to temperature/humidity	Weather Signature Database
9	Build baseline and price-only ML models	Baseline results
10	Build weather-aware model and ablation tests	Main ML comparison
11	Event-day evaluation and optional shock classifier	Event-focused results
12	Explainability + similar historical cases	SHAP + similarity module
13	Build and integrate Streamlit application	Working prototype
14	Final testing, documentation and presentation	Complete project package

20. Review Milestones

Review 1 — End of Week 4

- GitHub repository
- Cleaned price data
- Cleaned weather data
- Source register
- Master merged dataset
- Data dictionary
- Data Quality Report

Review 2 — End of Week 8

- Weather feature engine
- Weather event definitions
- Event-window module
- Rainfall signature analysis
- Temperature/humidity signature analysis
- Weather Signature Database

Review 3 — End of Week 11

- Baseline model results
- Price-only ML results
- Weather-aware ML results
- Ablation comparison
- All-day versus weather-event-day evaluation
- Optional price-shock classification results

Final Review — Week 14

- Explainable AI results
- Similar historical weather cases
- Working Streamlit application
- Reproducible repository
- Final technical report
- Presentation and demonstration

21. What You Must Not Do

- Do not randomly shuffle the time series for train/test splitting.
- Do not delete extreme prices or weather observations without verification.
- Do not create dozens of unexplained features.
- Do not compare many algorithms simply to find the best score.
- Do not make seasonality, festivals or auspicious days part of this project's main analysis.

- Do not claim that weather “caused” a price change simply because the variables are associated.
- Do not build the web application before the analytical pipeline is working.
- Do not report only overall model performance; event-day performance is essential.
- Do not hide negative results or model failures.

22. Final Deliverables

8. Cleaned daily price dataset.
9. Cleaned and documented daily weather dataset.
10. Merged weather–price master dataset.
11. Data dictionary and source register.
12. Research log documenting decisions and problems.
13. Weather feature-engineering module.
14. Weather event-window extraction module.
15. Weather Signature Database.
16. Visual analysis of rainfall, temperature and humidity event signatures.
17. Baseline, price-only and weather-aware ML models.
18. Ablation experiment showing the value added by weather information.
19. All-day and weather-event-day model evaluation.
20. Explainable AI analysis.
21. Similar historical weather-case retrieval.
22. Working Weather–Price Intelligence Streamlit application.
23. GitHub repository with reproducible notebooks/scripts and README.
24. Final student project report and presentation.

23. What Counts as a Successful Project?

A successful project is not defined by achieving the lowest possible prediction error. It is successful if the team can clearly demonstrate what information was used, how weather events were represented computationally, whether weather information adds measurable value, where the model performs well or poorly, and how the results can be explored transparently through a working prototype.

The project’s central technical contribution

Build a computational representation of short-term weather conditions and test whether those representations contain useful information about abnormal Shevanti price behaviour.

24. Starting Checklist

- ☐ Create the GitHub repository and agreed folder structure.
- ☐ Place raw price and weather files in a read-only raw-data folder.
- ☐ Create a data-source register.
- ☐ Complete the first data-quality checks.
- ☐ Agree on weather variables and units.

- ☐ Produce the first merged sample and verify dates manually.
- ☐ Prepare the Week 1–4 deliverables before attempting ML.